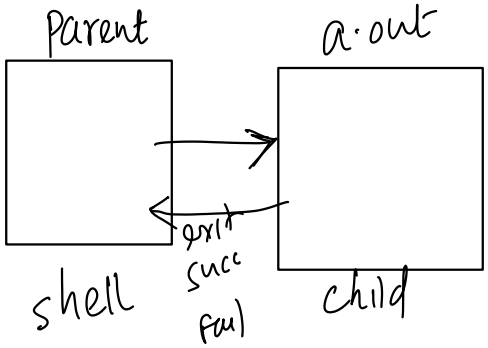
→ Normally in main function, we use return 0. why?
Although return 1 or 2 or 20 also behave the same in my App.

- return 0; in main is making program terminate.



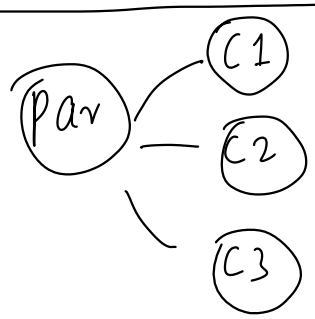

```
main() {
  ≡
  ≡
  ≡
  ≡
  ≡
  ≡
  return 0;
}
```

→ shell considers the exit-value 0 as Normal termination of child
success

Non-zero exit value ~~of~~ as a failure

→ exit values are ranged between (- 0 - 255)

→ In parent exit value can be collected by wait(), waitpid()

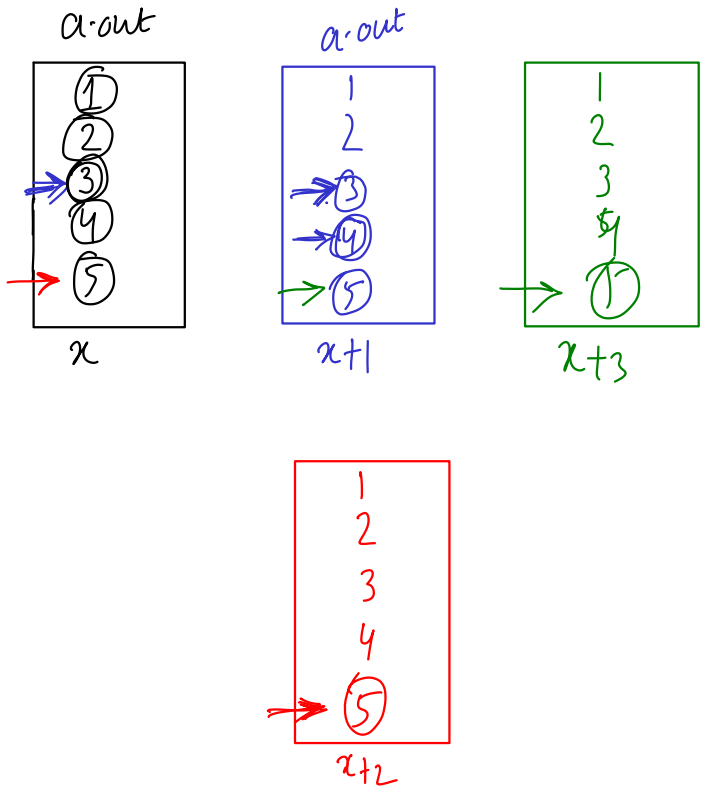



```
par() {
  fork();
  fork();
  fork();
  ≡
```

3

total No. of Jobs will be 8

```
main() {
    1 -
    2 - fork();
    3 -
    4 fork();
    5 -
}
```



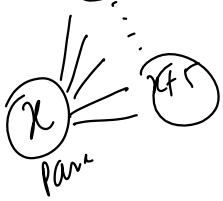

```
main() {

    for( i=1; i<=5; i++)
    {
        ≡
        fork();
        ≡
    }

}
```

Total 32 jobs will be made available

This type of multiple job creation NOT preferred

```
main() {
    for( i=1; i<=5; i++)
    {
        if( fork()==0) {
            ≡ | child code
            exit(0);
        }
    }
}
```

Total : 6



waitpid(-1 , &var, 0);

is same as

wait (&var);

waitpid(child2, &v, 0);

The parent will be blocked, until child 2 has not changed state.

waitpid(child2 ID, &v, WNOHANG);

↳ This flag makes the waitpid - non Block

What is the use of wait - without blocking the parent

par () {

2sec

}

Child {

5sec

}

Useful only when child is already a zombie, then waitpid - with WNOHANG called

Tomorrow : Signal Handling in Linux.

signal, Sigaction, alarm, etc--

raising of a signal, Delivery, ISR, user-defined ISR